

Basic (a simple version log) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

A version log is just a record of the different variants of one prompt you've tried, each one with a name ("v1" , "v2"), the actual prompt text, and when it was saved. Save that as a small list of dictionaries in a JSON file — nothing fancier than that.

Things to use:

- A dictionary for each version: `{"version_name": "v1", "prompt_text": "...", "created_at": "..."} .`
- A list to hold more than one version.
- `json.dump(data, file)` to write it, `json.load(file)` to read it back.
- `datetime.now()` for a timestamp.

Intermediate Version

Two real Python skills carry this whole exercise: writing/reading a JSON file, and building up a list of dicts one entry at a time. `json.dump(data, f, indent=2)` writes readable JSON; `indent=2` is optional but makes the file human-readable if you ever open it directly. `json.load(f)` reads it back into the exact same Python structure (list of dicts) you saved.

For the timestamp, `datetime.now(timezone.utc).isoformat()` gives you a sortable, unambiguous string like `"2026-09-11T14:03:00+00:00"` — always store it as UTC, not local time, so it means the same thing no matter who reads the log later.

The exact pieces:

- `import json` — `json.dump(data, f, indent=2)` and `json.load(f)` .
- `from datetime import datetime, timezone` — `datetime.now(timezone.utc).isoformat()` .
- `pathlib.Path(path).exists()` — check whether the log file already exists before deciding whether to start a new list or load the existing one.
- A stand-in for Doc13's test suite: a small function, `run_eval_suite(prompt_text)` , that runs a handful of test cases against the prompt and returns a score. In this exercise you can stub it (fake, deterministic answers) so the script runs on its own — in real Project 5 code, this calls your actual Doc13 suite.

Difference between Basic and Intermediate: Basic names the plain idea — save two versions, score each one. Intermediate explains why the log must load before it saves, why a damaged file must stop the save instead of becoming an empty list, and why the timestamp should be UTC and sortable.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
make an empty list called versions

make version v1: {"version_name": "v1", "prompt_text": "...", "created_at": now}
make version v2: {"version_name": "v2", "prompt_text": "...", "created_at": now}
add both to versions

save versions to versions.json with json.dump

for each version in versions:
    run the eval suite against its prompt_text
    print the version name and its score
```

Here's almost the whole thing — just try running it and reading it line by line:

```
# version_log_practice.py
import json
from datetime import datetime

def run_eval_suite(prompt_text):
    # stand-in for Doc13's real test suite
    if "polite" in prompt_text:
        return {"score": 1.0}
    return {"score": 0.6}

versions = []
versions.append({
    "version_name": "v1",
    "prompt_text": "Answer the customer's question.",
    "created_at": str(datetime.now()),
})
versions.append({
    "version_name": "v2",
    "prompt_text": (
        "Answer the customer's question politely and cite the source."
    ),
    "created_at": str(datetime.now()),
})
```

```
})

with open("versions.json", "w") as f:
    json.dump(versions, f, indent=2)

for version in versions:
    result = run_eval_suite(version["prompt_text"])
    print(version["version_name"], result["score"])
```

Expected output:

```
v1 0.6
v2 1.0
```

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
function save_version(name, prompt_text, path) -> None:
    load existing list from path, or start a new one if the file doesn't exist
    append {"version_name": name, "prompt_text": prompt_text,
           "created_at": utc now, isoformat}
    write the whole list back to path

function load_versions(path) -> list[dict]:
    if file doesn't exist: return empty list
    read and return the JSON list

save_version("v1", ..., "versions.json")
save_version("v2", ..., "versions.json")

for version in load_versions("versions.json"):
    run the eval suite, print name and score
```

```
# version_log_practice.py
import json
from datetime import datetime, timezone
from pathlib import Path

def save_version(name: str, prompt_text: str, path: str) -> None:
    versions = load_versions(path)
    versions.append({
        "version_name": name,
        "prompt_text": prompt_text,
        "created_at": datetime.now(timezone.utc).isoformat(),
    })
    with open(path, "w") as f:
        json.dump(versions, f, indent=2)

def load_versions(path: str) -> list[dict]:
    if not Path(path).exists():
        return []
    with open(path) as f:
        return json.load(f)
```

Now write the loop that calls `run_eval_suite` for each saved version and prints its score, and compare against the [Solution](#).

Difference between Basic and Intermediate: Basic builds the list in memory and writes it once. Intermediate splits the work into `save_version` / `load_versions` , so each save appends to what's already on disk, and a damaged file raises a named error.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)